Recurrent Neural Networks, Sequence to Sequence Models and Attention Mechanisms

George Michailidis

gmichail@ucla.edu

STAT 426

Training RNNs

Forward Pass vs. Backward Pass

- Forward: The RNN unrolls over time steps $t = 1 \dots T$, computing hidden states $\mathbf{h}_t$ and outputs $\mathbf{o}_t$ sequentially.
- Backward: To update weights (especially $\mathbf{W}_{hh}$), we must propagate error gradients from the future back into the past.

The Core Difficulty Unlike Feedforward Networks (MLP), the gradient at time t depends on computations from all previous time steps $1, \cdots, t-1$.

- This creates a dependency chain that grows linearly with the sequence length T.
- End Result: High memory usage and numerical instability (Vanishing/Exploding Gradients).

Illustration through a Simplified RNN

To rigorously analyze Backpropagation through Time (BPTT), we define a simplified RNN (no bias, identity activation).

Forward Pass Equations: For a single input $\boldsymbol{x}_t$ and target y_t at time step t:

$$\text{Hidden State: } \boldsymbol{h}_t = \boldsymbol{W}_{hx}\boldsymbol{x}_t + \boldsymbol{W}_{hh}\boldsymbol{h}_{t-1}$$

$$\text{Output: } \boldsymbol{o}_t = \boldsymbol{W}_{qh}\boldsymbol{h}_t$$

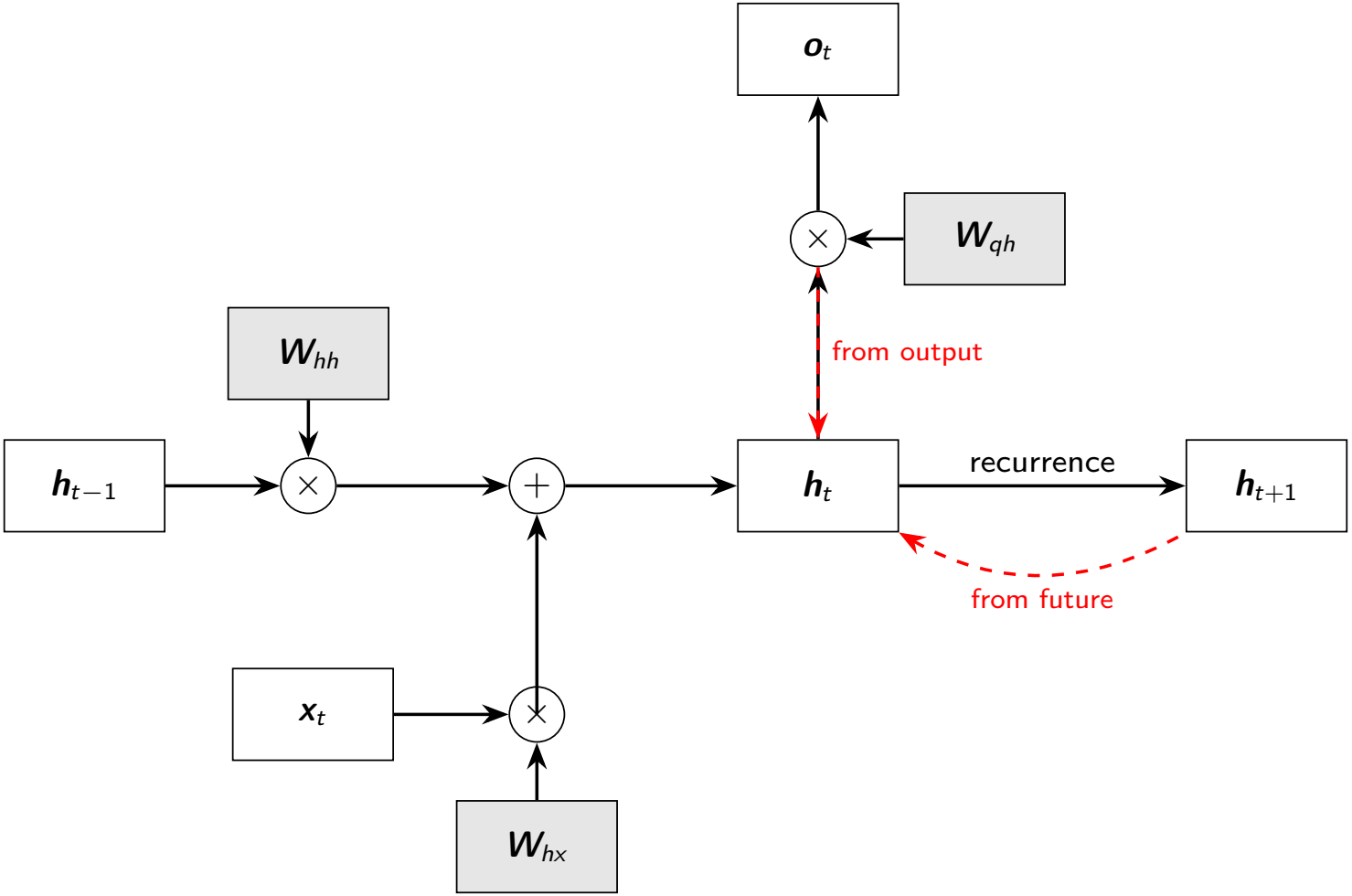
$$\text{Objective: } \mathcal{L} = \frac{1}{T}\sum_{t=1}^{T}\ell(\boldsymbol{o}_t, y_t)$$

The Key Challenge:

- Parameters $\boldsymbol{W}_{hx}, \boldsymbol{W}_{hh}, \boldsymbol{W}_{qh}$ are shared across all time steps.
- Computing gradients (backpropagation) requires unwinding the dependency chain $t = T \to 1$.
- We must differentiate $\mathcal{L}$ with respect to parameters that appear T times in the computational graph.

The Computational Graph

Training requires traversing the graph in reverse.



Key dependency: $\boldsymbol{h}_t$ affects the loss via:
(i) The immediate output $\boldsymbol{o}_t$ (Vertical flow); (ii) The future hidden state $\boldsymbol{h}_{t+1}$ (Horizontal flow)

The Loss Function

Let the objective function $\mathcal{L}$ be the average loss over T time steps.

Depending on the task, the step-wise loss $\ell(\mathbf{o}_t, y_t)$ is typically:

- Regression: ℓ_2 norm (MSE).
- Classification: Cross-Entropy.

$$\mathcal{L} = \frac{1}{T} \sum_{t=1}^{T} \ell(\mathbf{o}_t, y_t)$$

Goal: Minimize $\mathcal{L}$ by updating shared weights $\boldsymbol{W}_{hx}, \boldsymbol{W}_{hh}, \boldsymbol{W}_{qh}$.

Output Layer Gradients

We compute gradients in two stages: immediate outputs first, then recursive hidden states.

1. Gradient w.r.t Output $\boldsymbol{o}_t$: This is straightforward differentiation of the local loss function at step t:

$$\frac{\partial \mathcal{L}}{\partial \boldsymbol{o}_t} = \frac{1}{T} \frac{\partial \ell(\boldsymbol{o}_t, y_t)}{\partial \boldsymbol{o}_t}$$

2. Gradient w.r.t Output Weights $\boldsymbol{W}_{qh}$: Using the chain rule, summing contributions from every time step:

$$\frac{\partial \mathcal{L}}{\partial \boldsymbol{W}_{qh}} = \sum_{t=1}^{T} \frac{\partial \mathcal{L}}{\partial \boldsymbol{o}_t} \frac{\partial \boldsymbol{o}_t}{\partial \boldsymbol{W}_{qh}} = \sum_{t=1}^{T} \frac{\partial \mathcal{L}}{\partial \boldsymbol{o}_t} \boldsymbol{h}_t^\top$$

Note: Storing $\boldsymbol{h}_t$ for every t is required to compute this sum.

The Recursive Hidden Gradient

The gradient of the total loss with respect to the hidden state $\boldsymbol{h}_t$ is the sum of two terms (Chain Rule):

The Recurrent Gradient Equation

$$\frac{\partial \mathcal{L}}{\partial \boldsymbol{h}_t} = \underbrace{\frac{\partial \mathcal{L}}{\partial \boldsymbol{o}_t}\frac{\partial \boldsymbol{o}_t}{\partial \boldsymbol{h}_t}}_{\text{Immediate contribution}} + \underbrace{\frac{\partial \mathcal{L}}{\partial \boldsymbol{h}_{t+1}}\frac{\partial \boldsymbol{h}_{t+1}}{\partial \boldsymbol{h}_t}}_{\text{Future contribution}}$$

- Base Case ($t = T$): The "Future" term is 0 (no $\boldsymbol{h}_{T+1}$).
- Recursive Step ($t < T$):

$$\frac{\partial \mathcal{L}}{\partial \boldsymbol{h}_t} = \boldsymbol{W}_{qh}^\top \frac{\partial \mathcal{L}}{\partial \boldsymbol{o}_t} + \boldsymbol{W}_{hh}^\top \frac{\partial \mathcal{L}}{\partial \boldsymbol{h}_{t+1}}$$

This equation shows that to calculate the gradient at t, we must have already computed the gradient at $t + 1$.

Analytical Expansion & Instability

If we expand the recursive equation for $\frac{\partial \mathcal{L}}{\partial \boldsymbol{h}_t}$ backwards by $T - t$ steps, we see the accumulation of the weight matrix $\boldsymbol{W}_{hh}$:

$$\frac{\partial \mathcal{L}}{\partial \boldsymbol{h}_t} = \sum_{i=t}^{T} \left((\boldsymbol{W}_{hh}^\top)^{i-t} \boldsymbol{W}_{qh}^\top \frac{\partial \mathcal{L}}{\partial \boldsymbol{o}_i} \right)$$

Eigenvalue Analysis The term $(\boldsymbol{W}_{hh}^\top)^{i-t}$ involves raising a matrix to a high power.

- If eigenvalues $|\lambda| > 1$: **Exploding Gradients** (Values diverge).
- If eigenvalues $|\lambda| < 1$: **Vanishing Gradients** (Signal decays to 0).

Implication: This linear example shows why standard RNNs struggle with long sequences, necessitating LSTM/GRU (which alter this gradient flow) or Gradient Clipping.

Accumulating Parameter Gradients

Finally, we compute gradients for the shared parameters $\boldsymbol{W}_{hx}$ and $\boldsymbol{W}_{hh}$. Since these are reused at every step, we sum the contributions from all $\boldsymbol{h}_t$.

Final Parameter Updates

$$\frac{\partial \mathcal{L}}{\partial \boldsymbol{W}_{hx}} = \sum_{t=1}^{T} \frac{\partial \mathcal{L}}{\partial \boldsymbol{h}_t} \frac{\partial \boldsymbol{h}_t}{\partial \boldsymbol{W}_{hx}} = \sum_{t=1}^{T} \frac{\partial \mathcal{L}}{\partial \boldsymbol{h}_t} \boldsymbol{x}_t^\top$$

$$\frac{\partial \mathcal{L}}{\partial \boldsymbol{W}_{hh}} = \sum_{t=1}^{T} \frac{\partial \mathcal{L}}{\partial \boldsymbol{h}_t} \frac{\partial \boldsymbol{h}_t}{\partial \boldsymbol{W}_{hh}} = \sum_{t=1}^{T} \frac{\partial \mathcal{L}}{\partial \boldsymbol{h}_t} \boldsymbol{h}_{t-1}^\top$$

- Storage: We must cache $\boldsymbol{x}_t, \boldsymbol{h}_t, \boldsymbol{h}_{t-1}$ for the entire sequence (or truncated block).
- Compute: Computation of $\frac{\partial \mathcal{L}}{\partial \boldsymbol{h}_t}$ is the bottleneck, as it can not be parallelized (must wait for $t+1$).

Practical Solution: Truncated BPTT

Full BPTT is often computationally infeasible for very long sequences (e.g., long time series data, or long text passages) due to memory constraints and gradient instability.

The Strategy

1. **Split** the long sequence into shorter subsequences of length τ (e.g., $\tau \approx 35$).
2. **Detach** the computational graph at the subsequence boundaries.

Mechanism:

- Forward Pass: The final hidden state $\mathbf{h}_\tau$ of block k becomes the initial state $\mathbf{h}_0$ of block $k+1$. This preserves the *context*.
- Backward Pass: We stop the gradient flow at the boundary ($\frac{\partial \mathcal{L}}{\partial \mathbf{h}_\tau} \leftarrow 0$).

Trade-off: This limits the gradient's "memory" to τ steps, sacrificing long-term dependency learning for computational efficiency and stability.

Summary of Training Dynamics

1. **BPTT** is standard backpropagation applied to an unrolled time graph.
2. **Shared Weights:** The same $\mathbf{W}_{hh}$ is used at every step, meaning gradients from all steps accumulate into one update for $\mathbf{W}_{hh}$.
3. **Stability:** The recursive multiplication of matrices makes RNNs inherently unstable (Vanishing/Exploding gradients).
4. **Truncation:** We typically truncate the backward pass after τ steps to save memory and prevent divergence, treating the initial hidden state of the block as a constant.

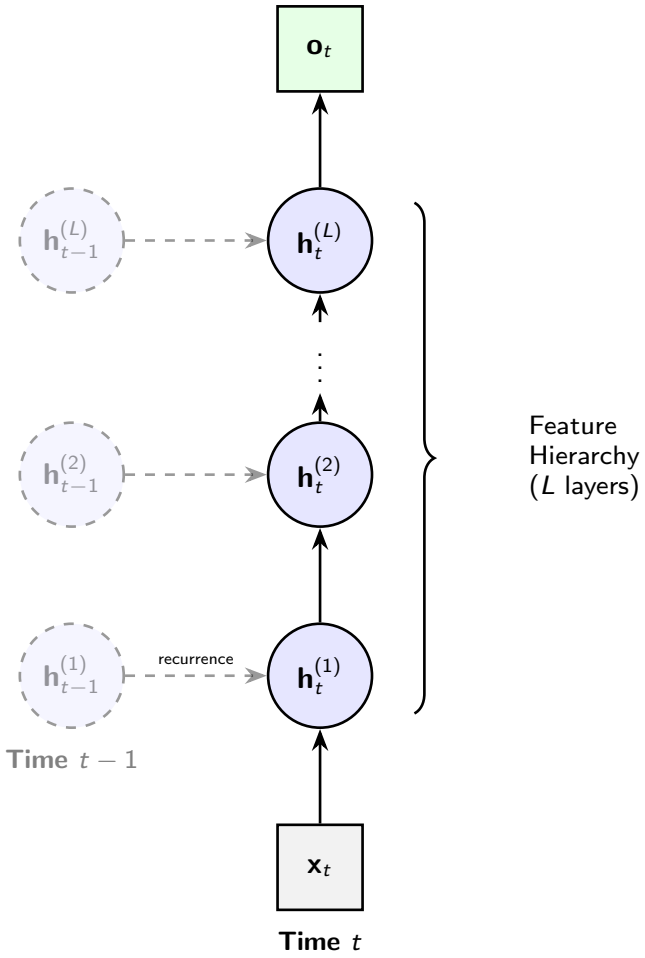
Motivation for Deep RNNs

- Standard RNNs typically have a **single hidden layer**.
- This implies only one linear transformation + activation per time step.
- Limitation: It is difficult to capture highly non-linear, hierarchical features (e.g., syntax $\rightarrow$ semantics $\rightarrow$ sentiment).

Going deep:

- Just as Deep CNNs stack layers to separate features (edges $\rightarrow$ shapes $\rightarrow$ objects), Deep RNNs stack multiple recurrent hidden layers.
- Benefit: Allows the network to process information at different time scales and levels of abstraction simultaneously.

Schematic of Deep RNN Architecture



Functional Dependencies

In a Deep RNN with L hidden layers, the hidden state at layer l depends on two sources of information:

1. Spatial Input (Bottom-Up): The output from the layer below $(l-1)$ at the *current* time step t.
2. Temporal Input (Left-Right): The state of the same layer (l) at the *previous* time step $t-1$.

The Core Equation for Layer l

$$\mathbf{h}_t^{(l)} = \phi_l \left(\underbrace{\mathbf{W}_{xh}^{(l)} \mathbf{h}_t^{(l-1)}}_{\text{From Below}} + \underbrace{\mathbf{W}_{hh}^{(l)} \mathbf{h}_{t-1}^{(l)}}_{\text{From Past}} + \mathbf{b}_h^{(l)} \right)$$

Note: For the first layer ($l = 1$), the input is the raw data $\mathbf{x}_t$.

Parameters and Output

Parameter Independence

- Each layer l has its own unique weights:
  - $\mathbf{W}_{xh}^{(l)}$ (Vertical Weights)
  - $\mathbf{W}_{hh}^{(l)}$ (Horizontal Weights)
  - Bias $\mathbf{b}_h^{(l)}$
- Weights are not shared between layers (Vertical).
- Weights are shared across time steps (Horizontal).

The Output Layer

- Only the top layer $\mathbf{h}_t^{(L)}$ connects to the final output.
- The lower layers act purely as feature extractors.

Final Prediction

$$\mathbf{o}_t = \mathbf{W}_{hq}\mathbf{h}_t^{(L)} + \mathbf{b}_q$$

Implementation & Summary

Computational Cost: Training Deep RNNs requires backpropagation along two axes:

1. Through Time: Gradients flow backward along the unrolled time dimension (often truncated via Truncated BPTT).
2. Through Depth: Gradients flow down through stacked layers (from layer L to 1).

Scaling: Costs increase with sequence length T, batch size B, hidden size H, and depth L. Memory usage is dominated by storing activations at every step t and layer l for the backward pass.

Notes on implementation:
While libraries handle the autograd math and provide `num_layers`, the data scientist must explicitly manage:

- **Truncation:** Deciding the window size τ for BPTT.
- **Variable Lengths:** Using `pack_padded_sequence` to mask padding tokens efficiently.
- **State Management: Detaching** hidden states between batches (‘h.detach()‘) to prevent the graph from growing infinitely and consuming all memory.

Implementation Challenges: Padding & States

1. Variable Sequence Lengths

The Issue: Real data (sentences) have different lengths. To fit them in a batch matrix, we pad short sequences with zeros (0).

$$\begin{bmatrix} \text{Hi} & 0 & 0 \\ \text{How} & \text{are} & \text{you} \end{bmatrix}$$

The Problem: The RNN processes the zeros as real data, corrupting the hidden state with "noise."

Solution: `pack_padded_sequence`: Tells the RNN the *actual* length of each sample so it skips the zeros.

2. State Management (Detaching)

The Issue: We process long text in chunks (batches). We want to pass the *hidden state* to the next batch to maintain context.

The Problem: PyTorch tries to backpropagate through the entire history (Batch $10 \to 9 \to \cdots \to 1$), causing Out of Memory errors.

Solution:
`h.detach()`: Passes the *values* of the hidden state to the next batch, but cuts the *gradient history*.

The Limitations of Standard RNNs

So far, we have discussed building simple/deep RNNs (LSTM/GRU) for one-to-many, one-to-one or many-to-one tasks.

However, in In standard RNN formulations, the input sequence length T_x and output sequence length T_y usually share a rigid relationship (often $T_x = T_y$ or $T_y = 1$).

The Problem (Sequence-to-Sequence):

- Natural Language Processing:
  - Source: "Hello world" ($T_x = 2$)
  - Target: "Bonjour le monde" ($T_y = 3$)
- Time Series Forecasting:
  - Input: Past 24 hours of sensor data ($T_x = 24$)
  - Output: Future 48 hours of predictions ($T_y = 48$)

Here, $T_x \neq T_y$, and the alignment is non-monotonic. An MLP cannot handle this because the input dimension is variable. A standard Many-to-Many RNN can not handle this because the output length is not known *a priori*.

The Encoder-Decoder Paradigm

To solve the variable-length mapping problem, two distinct sub-networks are introduced and jointly trained:

1. The Encoder: A function f_{enc} that reads the variable-length input sequence $\mathbf{x}_1, \cdots, \mathbf{x}_{T_x}$ and compresses it into a fixed-length internal representation.
2. The Context Variable ($\mathbf{c}$): The fixed-length vector (or set of vectors) that encapsulates the *entire* semantic information of the input sequence. This is the information bottleneck.
3. The Decoder: A function f_{dec} that takes the context variable $\mathbf{c}$ and generates the variable-length output sequence $\mathbf{y}_1, \cdots, \mathbf{y}_{T_y}$, one token at a time.

3. The Encoder: Hidden State Dynamics

Let the input sequence be $\mathcal{X} = \{\mathbf{x}_1, \ldots, \mathbf{x}_T\}$, where T is the input length.

The Encoder is an RNN (or LSTM/GRU). At any time step $t \in [1, T]$, the Encoder updates its hidden state $\mathbf{h}_t$.

Encoder Update Equation

$$\mathbf{h}_t = f(\mathbf{x}_t, \mathbf{h}_{t-1})$$

- $\mathbf{x}_t$: The input feature vector (e.g., word embedding) at time t.
- $\mathbf{h}_{t-1}$: The hidden state from the previous time step.
- f: A non-linear recurrent function (e.g., a GRU cell).

Note: This process is identical to a standard RNN language model. We simply process the input until the end of the sequence.

The Encoder: The Context Variable $\mathbf{c}$

After processing the entire input sequence, we must define the Context Variable $\mathbf{c}$. The variable $\mathbf{c}$ serves as the "summary" of the input. It is constructed via a function q applied to the Encoder's hidden states:

$$\mathbf{c} = q(\mathbf{h}_1, \ldots, \mathbf{h}_T)$$

Common Instantiations of q

1. Last State (Most Common): We assume the final hidden state captures the complete sequence history.
$$\mathbf{c} = \mathbf{h}_T$$
2. Transformation: An MLP applied to the last state.
$$\mathbf{c} = \tanh(\mathbf{W}\mathbf{h}_T)$$

Crucially, $\mathbf{c}$ *has a fixed dimension regardless of* T*.*

The Decoder: State Initialization

The Decoder is a separate RNN. We denote its hidden states as $\mathbf{s}_{t'}$ (using index t' to distinguish output time from input time t).

The Decoder must be initialized. Unlike a standard RNN where $\mathbf{h}_0$ is often $\mathbf{0}$, the Decoder's initial state $\mathbf{s}_0$ is typically conditioned on the Context Variable $\mathbf{c}$.

$$\mathbf{s}_0 = g(\mathbf{c})$$

Often, we simply set $\mathbf{s}_0 = \mathbf{c}$ (if dimensions match).

This effectively "injects" the meaning of the source sentence into the Decoder before it generates the first token.

The Decoder: Generative Recurrence

At output time step t', the Decoder updates its hidden state $\mathbf{s}_{t'}$ and predicts the next token.

The state $\mathbf{s}_{t'}$ depends on:

1. The previous hidden state $\mathbf{s}_{t'-1}$.
2. The previous generated token $\mathbf{y}_{t'-1}$ (or the ground truth during training).
3. (Optionally) The context variable $\mathbf{c}$ at every step.

Decoder Update Equation

$$\mathbf{s}_{t'} = g(\mathbf{y}_{t'-1}, \mathbf{c}, \mathbf{s}_{t'-1})$$

Note the explicit inclusion of $\mathbf{c}$. In basic implementations, $\mathbf{c}$ might only be used to set $\mathbf{s}_0$, but in modern implementations, it is often concatenated to the input at every time step t'.

Digression: What is a "Token"? (Discrete vs. Continuous)

In the context of Encoder-Decoder models, a Token ($\mathbf{y}_{t'}$) is simply the atomic unit of the sequence generated at each step.

1. NLP (Text)

- **Unit:** A word ("cat"), a sub-word ("ing"), or a character ("a").
- **Nature: Discrete** (Categories).
- **Vocabulary:** Fixed size V (e.g., 30,000 words).
- **Representation:** One-hot vector or Embedding index.
- **Output Layer:**

$$P(\mathbf{y}_{t'}) = \text{Softmax}(\mathbf{W}\mathbf{s}_{t'})$$

(Classifying which word comes next).

2. Time Series Data

- **Unit:** A scalar (Temperature at 10:00 AM) or a vector (Stock High/Low/Close).
- **Nature: Continuous** (Real numbers).
- **Vocabulary:** None (Infinite possibilities).
- **Representation:** Real-valued vector $\in \mathbb{R}^d$.
- **Output Layer:**

$$\hat{\mathbf{y}}_{t'} = \text{Linear}(\mathbf{s}_{t'})$$

(Regressing the exact numerical value).

The Output Probability Distribution

The goal is to model the conditional probability of the target sequence $\mathcal{Y}$ given the source $\mathcal{X}$.

The output layer of the Decoder (typically a Linear layer + Softmax) computes the probability of the token $y_{t'}$:

$$P(y_{t'} \mid y_1, \ldots, y_{t'-1}, \mathbf{c}) = \text{softmax}(\mathbf{W}_o \mathbf{s}_{t'} + \mathbf{b}_o)$$

Formal Objective: By the Chain Rule of Probability, the probability of the entire sequence $\mathbf{y}$ is:

$$P(\mathbf{y} \mid \mathbf{x}) = \prod_{t'=1}^{T'} P(y_{t'} \mid y_1, \ldots, y_{t'-1}, \mathbf{c})$$

Training Strategy: Teacher Forcing

There is a critical distinction in how we feed $\mathbf{y}_{t'-1}$ into the Decoder.

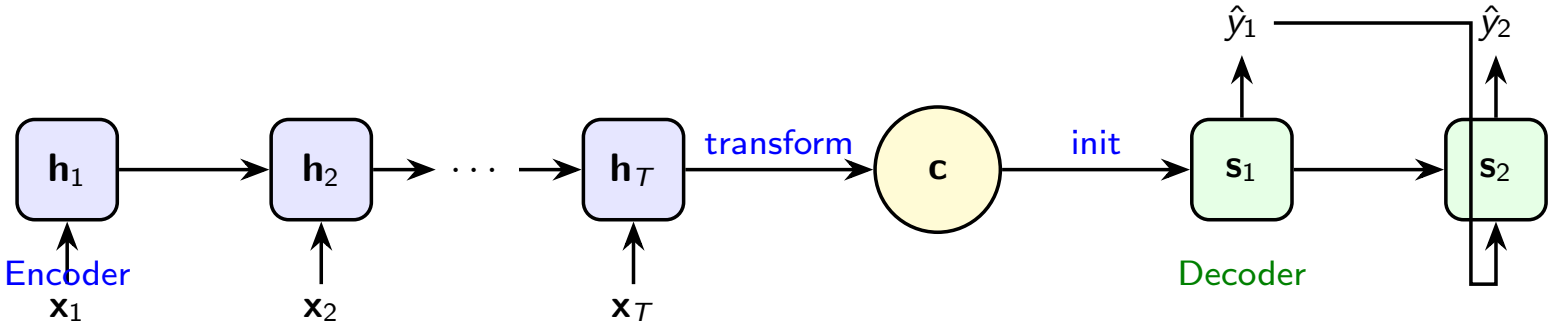
Training (Teacher Forcing)

- We know the ground truth target sequence.
- Input at step t' is the actual ground truth token $y_{t'-1}$.
- This prevents errors from compounding early in training.

Inference (Test Time)

- We do not know the target.
- Input at step t' is the predicted token $\hat{y}_{t'-1}$ (argmax or sampled) from the previous step.
- This is auto-regressive generation.

9. Architecture Diagram



Summary and The Bottleneck Problem

Summary of Mechanisms

1. Encoding: $\mathbf{x}_{1\ldots T} \to \text{RNN} \to \mathbf{h}_T \to \mathbf{c}$.
2. Decoding: $(\mathbf{c}, \mathbf{y}_{<t'}) \to \text{RNN} \to \mathbf{s}_{t'} \to \mathbb{P}(y_{t'})$.
3. Loss: Cross-Entropy/ℓ_2 summed over output sequence length T'.

The Theoretical Limitation: The Context Variable $\mathbf{c}$ is a fixed-size vector. It must encode the nuance of a sentence regardless of whether the sentence has 5 words or 100 words, or the time series is long or short.

This creates an Information Bottleneck.
As T increases, the ability of $\mathbf{h}_T$ to retain information from $\mathbf{x}_1$ (the start of the sequence) diminishes due to the fixed capacity and vanishing gradients. This limitation necessitates the Attention Mechanism (future topic).

Breaking the Context Bottleneck

Recap: The Encoder-Decoder Challenge: In standard Seq2Seq, the Encoder must compress the *entire* source sequence into a single fixed-length vector $\mathbf{c}$.

The Problem:

- **Information Loss:** For long sequences (e.g., many words, observations), $\mathbf{c}$ acts as a bottleneck, forgetting early details.
- **Path Length:** To access information from $\mathbf{x}_1$ at the end of generation, the gradient must flow back through T time steps (vanishing gradient risk).

The Solution (Attention): Instead of a static $\mathbf{c}$, we generate a dynamic context vector $\mathbf{c}_{t'}$ for *every* decoding step t'.

This creates Direct Connections between the Decoder state and specific Encoder states, reducing the path length from $O(T)$ to $O(1)$.

Theoretical Origins: Volitional vs. Non-Volitional

The concept of "Attention" is borrowed from cognitive neuroscience. We distinguish two types of selection:

1. Non-Volitional Cues (Keys)
   - **Mechanism:** Saliency.
   - Elements that "stand out" naturally (e.g., a red object in a dark room).
   - In the attention model: **Keys** (**k**) — the intrinsic features of the input.
2. Volitional Cues (Queries)
   - **Mechanism:** Intent.
   - Conscious focus (e.g., "I am looking for a book").
   - In our model: **Query** (**q**) — the current state of the decoder.

Definition Attention is the mechanism of biasing the selection of **Keys** based on a **Query**.

The Core Abstraction: Differentiable Lookup

We formalize this using a Database Metaphor.

Consider a database $\mathcal{D} = \{(\mathbf{k}_1, \mathbf{v}_1), \ldots, (\mathbf{k}_m, \mathbf{v}_m)\}$.

- **Query (q):** What we are looking for (Decoder state $\mathbf{s}_{t-1}$).
- **Key (k):** How data is indexed (Encoder states $\mathbf{h}_1 \ldots \mathbf{h}_m$).
- **Value (v):** The actual content (usually same as Key, $\mathbf{h}_1 \ldots \mathbf{h}_m$).

Note on Notation: Here, m represents the Source Sequence Length (often denoted as T or T_x). The database contains one key-value pair for every token in the input.

Generalized Attention Definition: Unlike a Hash Map (Hard Selection), Attention returns a weighted average (Soft Selection):

$$\text{Attention}(\mathbf{q}, \mathcal{D}) = \sum_{i=1}^{m} \alpha(\mathbf{q}, \mathbf{k}_i)\mathbf{v}_i$$

Note: α is a probability weight ($0 \leq \alpha \leq 1, \sum \alpha = 1$).

The Calculation Mechanism (Step-by-Step)

How do we compute the context vector $\mathbf{c}$ for a single query $\mathbf{q}$?

1. Score (Alignment): Compute a scalar score e_i measuring similarity between $\mathbf{q}$ and every key $\mathbf{k}_i$.
$$e_i = \text{score}(\mathbf{q}, \mathbf{k}_i)$$
2. Normalize (Softmax): Convert raw scores into probabilities.
$$\alpha_i = \frac{\exp(e_i)}{\sum_j \exp(e_j)}$$
3. Aggregate: Compute the weighted sum of values.
$$\mathbf{c} = \sum_i \alpha_i \mathbf{v}_i$$

Scoring Function 1: Additive Attention

Use Case: When query $\mathbf{q}$ and key $\mathbf{k}$ have different dimensions.

$$e_i = \mathbf{v}_a^\top \tanh(\mathbf{W}_q \mathbf{q} + \mathbf{W}_k \mathbf{k}_i)$$

- We project $\mathbf{q}$ and $\mathbf{k}$ into a shared space using $\mathbf{W}_q$ and $\mathbf{W}_k$.
- We apply a non-linearity (tanh).
- We collapse to a scalar using a learned vector $\mathbf{v}_a$.

Pros: Flexible.
Cons: Computationally expensive (tanh + separate projections).

Scoring Function 2: Scaled Dot-Product

Use Case: When $\mathbf{q}$ and $\mathbf{k}$ have the same dimension d.

$$e_i = \frac{\mathbf{q}^\top \mathbf{k}_i}{\sqrt{d}}$$

Why divide by $\sqrt{d}$?

Assume $\mathbf{q}$ and $\mathbf{k}$ elements are independent random variables with mean 0, Var 1.

- The dot product sum $\sum_{j=1}^{d} q_j k_j$ has variance d.
- For large d (e.g., 512), scores become huge (e.g., 20 or -20).
- Result: Softmax gradients vanish (output is 1 or 0).
- Fix: Dividing by $\sqrt{d}$ rescales variance back to 1.

Vectorization: The Matrix Form

In practice (GPUs), we do not loop $i = 1, \cdots, m$. We compute all scores at once.

Let $\mathbf{Q} \in \mathbb{R}^{n \times d}$ (n queries) and $\mathbf{K} \in \mathbb{R}^{m \times d}$ (m keys).

$$\text{Attention}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{softmax}\left(\frac{\mathbf{Q}\mathbf{K}^\top}{\sqrt{d}}\right)\mathbf{V}$$

Dimensions Check:

- $\mathbf{Q}\mathbf{K}^\top \to (n \times d) \times (d \times m) = (n \times m)$ matrix of scores.
- Softmax is applied row-wise (for each query).
- Result $\times \mathbf{V} \to (n \times m) \times (m \times d_v) = (n \times d_v)$.

Result: For every query, we get a weighted sum of values.

Illustrative Numerical Example

Setup: 3 Keys/Values in $\mathbb{R}^2$: $\mathbf{k}_1 = [1, 0], \mathbf{k}_2 = [0, 1], \mathbf{k}_3 = [1, 1]$. Query $\mathbf{q} = [1, 0]$.

1. Dot Product Scores (Similarity):
   - $s_1 = \mathbf{q} \cdot \mathbf{k}_1 = 1 \times 1 + 0 \times 0 = \mathbf{1}$
   - $s_2 = \mathbf{q} \cdot \mathbf{k}_2 = 1 \times 0 + 0 \times 1 = \mathbf{0}$
   - $s_3 = \mathbf{q} \cdot \mathbf{k}_3 = 1 \times 1 + 0 \times 1 = \mathbf{1}$
2. Softmax ($e^x / \sum$):
   - Exponentials: $e^1 \approx 2.7, \quad e^0 = 1, \quad e^1 \approx 2.7$. Sum ≈ 6.4.
   - Weights α: $[0.42, 0.16, 0.42]$.
3. Weighted Sum:

$$\mathbf{c} = 0.42[1, 0] + 0.16[0, 1] + 0.42[1, 1] \approx [0.84, 0.58]$$

Interpretation: The query "attended" mostly to keys 1 and 3.

Attention in the Encoder-Decoder Loop

How does this all fit into the translation model we started with?

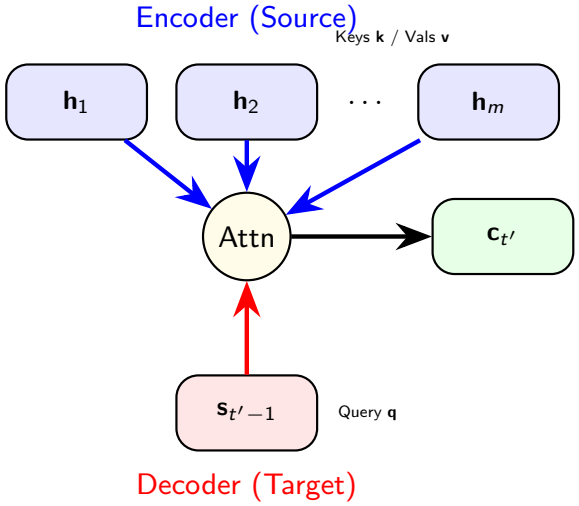
1. **The Setup (Encoder):** The Encoder processes source words ("Hello", "World") into hidden states $\mathbf{h}_1, \mathbf{h}_2$. $\Rightarrow$ These become the Keys ($\mathbf{K}$) and Values ($\mathbf{V}$).
2. **The Trigger (Decoder Step t'):** The Decoder has just generated "Bonjour" and has state $\mathbf{s}_{t'-1}$. $\Rightarrow$ This becomes the Query ($\mathbf{q}$).
3. **The Attention Step:** We calculate similarity between $\mathbf{s}_{t'-1}$ and all $\mathbf{h}$'s.

$$\alpha_i = \text{softmax}(\text{score}(\mathbf{s}_{t'-1}, \mathbf{h}_i))$$

4. **The Context ($\mathbf{c}_{t'}$):** We create a weighted sum $\mathbf{c}_{t'} = \sum \alpha_i \mathbf{h}_i$.
5. **The Prediction:** The Decoder uses $[\mathbf{s}_{t'-1}; \mathbf{c}_{t'}]$ to predict the next word ("le").

Key Takeaway: *The Decoder no longer blindly relies on a single static vector. It actively "searches" the source sentence for relevant information at every step (cross-attention).*

Visualization of Cross-Attention



Another Example for Time Series (Capturing Seasonality)

Consider forecasting **Electricity Usage** based on the last 7 days (168 hours).

1. **The Setup (Encoder):** The Encoder processes the sequence of past hourly loads ($\mathbf{x}_1 \dots \mathbf{x}_{168}$). $\Rightarrow$ These hidden states become the Keys ($\mathbf{K}$) and Values ($\mathbf{V}$).
2. **The Trigger (Decoder Step t'):** We are predicting usage for **8:00 AM tomorrow**. The Decoder has the current state $\mathbf{s}_{t'-1}$. $\Rightarrow$ This is the Query ($\mathbf{q}$) ("What looks like 8:00 AM?").
3. **The Attention Step:** The model compares the Query against history.
   - Comparison with 7:00 AM (Lag 1): Moderate Score.
   - Comparison with 8:00 AM yesterday (Lag 24): **High Score** (Daily Seasonality).
4. **The Context ($\mathbf{c}_{t'}$):** The context vector becomes a weighted average of *previous mornings*, ignoring the nights.
5. **The Prediction:** The Decoder uses this "summary of mornings" to predict the scalar value (kW).

Key Takeaway: *In Time Series, Attention automatically learns "Lag" dependencies (Weekly/Daily patterns) without us manually specifying them.*

From Cross-Attention to Self-Attention

So far, Query came from the Decoder, Key/Value from the Encoder ("Cross-Attention").

Self-Attention (the Transformer Core)
What if Query, Key, and Value all come from the same sequence $\mathbf{x}$?

$$\mathbf{q}_i = \mathbf{W}_q \mathbf{x}_i, \quad \mathbf{k}_i = \mathbf{W}_k \mathbf{x}_i, \quad \mathbf{v}_i = \mathbf{W}_v \mathbf{x}_i$$

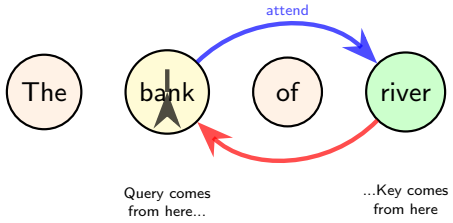
where

- i: The index of the token in the sequence $(1, \cdots, T)$.
- $\mathbf{x}_i$: The input vector for the word at position i (e.g., "bank").

Implication: Every token in the sentence looks at every other token to figure out context.

- Example: In "The **bank** of the river", the word "bank" attends to "river" to know it's not a financial bank.

Visualization of Self-Attention



Positional Encodings: Injecting Order

The Problem with Self-Attention:

$$\text{Attention}(\text{"I"}, \{\text{"hit"}, \text{"him"}\}) == \text{Attention}(\text{"I"}, \{\text{"him"}, \text{"hit"}\})$$

Self-Attention is **Permutation Invariant**. It treats the sentence as a "Bag of Words." The order of tokens does not change the result. [cite: 308, 309]

The Solution: We must explicitly add position information to the input embeddings.

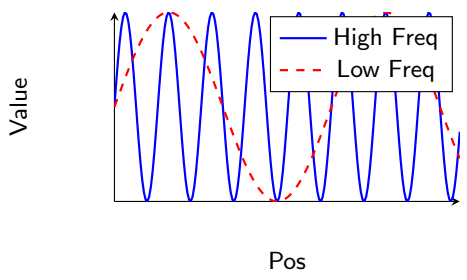
$$\mathbf{x}'_{pos} = \mathbf{x}_{pos} + \mathbf{PE}_{pos}$$

Sinusoidal Encoding (Fixed): Use sine/cosine waves of different frequencies.

$$PE_{(pos,2i)} = \sin(pos/10000^{2i/d})$$

$$PE_{(pos,2i+1)} = \cos(pos/10000^{2i/d})$$

(Where d is the model dimension and i is the dimension index)



High freq captures local order (e.g., adjacent words). Low freq captures global position (e.g., start vs. end).

Multi-Head Attention: The Intuition

Why do we need multiple "Heads"?

1. NLP Analogy (Text) When reading, you focus on different things simultaneously:

- **Head 1 (Syntax):** Finding the verb for this noun.
- **Head 2 (Semantics):** Linking "bank" to "river".
- **Head 3 (Tone):** Is this adjective positive?

2. Time Series Analogy When forecasting stock prices, you look for different patterns:

- **Head 1 (Short-term):** Yesterday's price impact.
- **Head 2 (Seasonality):** The price exactly 1 year ago.
- **Head 3 (Correlation):** The price of Oil or Gold.

The Problem: A single attention mechanism creates *one* average probability distribution. It cannot focus on "Yesterday" AND "Last Year" effectively if they are far apart.
The Solution: Run H attention layers in parallel. Each "Head" specializes in a different type of relationship.

The Math of Multi-Head Attention

Instead of one large attention, we project inputs into H smaller subspaces.

For each head $h = 1, \cdots, H$:

$$\text{head}_h = \text{Attention}(\mathbf{Q}\mathbf{W}_h^Q, \mathbf{K}\mathbf{W}_h^K, \mathbf{V}\mathbf{W}_h^V)$$

Concatenation: We combine the results from all heads and project back to the original dimension:

$$\text{MultiHead}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{Concat}(\text{head}_1, \ldots, \text{head}_H)\mathbf{W}^O$$

where $\mathbf{W}^O$ is the Output Projection Matrix.

Note: If model size $d = 512$ and heads $H = 8$, each head operates on dimension 64. The computation cost is roughly similar to single-head attention.

More on the Output Projection $\mathbf{W}^O$

After computing H separate attention heads, we concatenate them:

$$\mathbf{Z}_{\text{concat}} = \text{Concat}(\text{head}_1, \ldots, \text{head}_H)$$

If each head has dimension $d_v = 64$ and we have $H = 8$ heads, the concatenated vector has size 512. However, this is just a raw stack of independent features. We need to mix them.

We apply a final linear transformation:

$$\text{MultiHeadOutput} = \mathbf{Z}_{\text{concat}}\mathbf{W}^O$$

- **Dimensions:** $\mathbf{W}^O \in \mathbb{R}^{(H \cdot d_v) \times d_{model}}$ (e.g., 512×512).
- **Purpose:** It combines the insights from different heads (Syntax + Semantics + Tone) into a single unified representation for the next layer.

Dimensions in Practice: A Time Series Example

Let's trace the dimensions for a model forecasting Electricity Load.

1. The Input ($\mathbf{x}_t$): Raw input is low-dimensional (e.g., Load, Temp, Hour).

$$\mathbf{x}_t \in \mathbb{R}^3$$

2. The Embedding ($d_{model} = 512$): We project this small vector into a high-dimensional space to capture complex features.

$$\mathbf{h}_t = \text{Linear}(\mathbf{x}_t) \in \mathbb{R}^{512}$$

3. Splitting into Heads ($H = 8$): We don't want 8 giant matrices. We split the 512 features into 8 chunks of 64.

$$512 \div 8 = 64 \quad (d_k, d_v = 64)$$

- **Head 1:** Uses features 0-63 (e.g., focuses on Daily patterns).
- **Head 2:** Uses features 64-127 (e.g., focuses on Temperature correlations).
- ...

4. The Output: Each head produces a 64-dim vector. We concatenate them back:

$$[64] \times 8 \to 512$$

Finally, $\mathbf{W}^O$ mixes them: $512 \xrightarrow{\mathbf{W}^O} 512$.

Digression: Linear Projection vs. Lookup

How do we turn a tiny vector (size 3) into a giant vector (size 512)?

1. NLP (Discrete Tokens)
   - **Input:** An Integer Index (e.g., word ID "450").
   - **Mechanism: Lookup Table**.
   - We simply "grab" row #450 from a learned matrix $\mathbf{E} \in \mathbb{R}^{V \times d}$.
   - No multiplication involved (it's an indexing operation).

2. **Time Series (Continuous)**
   - **Input:** A Vector of Floats (e.g., $[23.5, 0.9, 14]$).
   - **Mechanism: Linear Layer**.
   - We multiply by a learnable weight matrix $\mathbf{W}_{emb}$.

The math of a Linear Projection: Given input $\mathbf{x}_t \in \mathbb{R}^3$ (Load, Temp, Hour) and target dimension $d = 512$:

$$\mathbf{h}_t = \mathbf{x}_t \mathbf{W}_{emb} + \mathbf{b}_{emb}$$

Dimensions: $[1 \times 512] = [1 \times 3] \cdot [3 \times 512] + [1 \times 512]$

Why? This "mixes" the raw features (e.g., combining "High Temp" and "Afternoon") into 512 rich, learnable features that the Attention heads can later split up and analyze.

Implementation Detail: Padding Masks

Problem: Batches contain sentences of different lengths. We pad short sentences with zeros.

$$[\text{"Hi"}, \text{"There"}, \text{PAD}, \text{PAD}]$$

Issue: We don't want the model to "attend" to PAD tokens (they have no meaning).

Solution (Masking): Before Softmax, we set the scores of PAD positions to $-\infty$.

$$\text{Softmax}([0.5, 0.8, \underbrace{-\infty}_{\text{PAD}}, \underbrace{-\infty}_{\text{PAD}}]) \rightarrow [0.42, 0.58, \mathbf{0}, \mathbf{0}]$$

This forces the attention weight to be exactly zero.

Implementation Detail: Causal Masks

Context: In the Decoder (during training), we feed the entire correct sentence.

Problem: When predicting word 3, the model shouldn't "see" word 4 (that's cheating/leaking the future).

Solution (Look-Ahead / Causal Mask): We apply a triangular mask to the score matrix.

$$\begin{bmatrix} s_{11} & -\infty & -\infty \\ s_{21} & s_{22} & -\infty \\ s_{31} & s_{32} & s_{33} \end{bmatrix}$$

Row 1 can only see Col 1. Row 2 can see Col 1 & 2. Future is hidden.

Summary & Complexity Analysis

Comparison: RNN vs. Transformer (Self-Attention) - Let T be the sequence length and d the dimension.

RNN

- **Complexity:** $O(T \cdot d^2)$
- **Parallelism:** $O(1)$ (Sequential)
- **Path Length:** $O(T)$ (Long dependencies are hard)

Self-Attention

- **Complexity:** $O(T^2 \cdot d)$
- **Parallelism:** $O(T)$ (Fully parallel)
- **Path Length:** $O(1)$ (Direct access to everything)

Attention is faster to train (parallelizable) and better at long-range dependencies, but memory grows quadratically with sequence length (T^2).